

#Program 1: Factorial of a Number

```
def fact(n):  
    if n<=1:  
        return n  
    else:  
        return n*fact(n-1)  
fact(5)
```

120

#Program 2: Fibonacci Series

```
def fib(n):  
    if n<=1:  
        return n  
    else:  
        return fib(n-1)+fib(n-2)  
for i in range(10):  
    print(fib(i))
```

0
1
1
2
3
5
8
13
21
34

#Program 3: Sum of First N Natural Numbers

```
def sum_n(n):  
    if n<=1:  
        return n  
    else:  
        return n+sum_n(n-1)  
sum_n(10)
```

55

find prime number or not

```
def prime_number(n):  
    if n<=1:  
        return False  
    for i in range(2,int(n**0.5)+1):  
        if n%i==0:  
            return False  
    return True  
prime_number(18)
```

False

Q1. How do you reverse a list?

```
l1=[3,1,5,2,7,5]
l2=l1[::-1]
print(f"Old list : {l1}")
l2
print(f"Old list : {l1}")

Old list : [3, 1, 5, 2, 7, 5]
Old list : [3, 1, 5, 2, 7, 5]
```

List

#Q2. Difference between append() and extend()?

```
lst = [1, 2, 3]

lst.append([4, 5])    # adds as single element → [1, 2, 3, [4, 5]]
print(lst)
lst.extend([4, 5])    # adds each element → [1, 2, 3, 4, 5]
print(lst)

[1, 2, 3, [4, 5]]
[1, 2, 3, [4, 5], 4, 5]
```

#Q3. Remove duplicates from a list while preserving order?

```
lst=[2,3,4,1,2,3,7,8]
s={}
result=[]
for i in lst:
    if i not in s:
        s[i]=True
        result.append(i)
print(result)
```

```
[2, 3, 4, 1, 7, 8]
```

Q4.Remove duplicates from a list using set?

```
lst=[2,3,4,1,2,3,7,8]
s=list(set(lst))
s
```

```
[1, 2, 3, 4, 7, 8]
```

#Q5. Find the second largest number in a list?

```
lst=[5,2,8,3,9,1,7]
lst.sort(reverse=True)
lst[2]
```

7

Swap two variables

```
x,y=9,8
print(f"Before swap x:{x}, y:{y}")
x,y=y,x
print(f"After swap x:{x},y:{y}")
```

Before swap x:9, y8

After swap x:8,y:9

Merge two dictionaries?

```
dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 3, 'c': 4}
print(dict1 | dict2)
```

{'a': 1, 'b': 3, 'c': 4}

Count frequency of elements (word count example)

```
lst=["Hello","Randhir","How","Are","You"]
from collections import Counter
freq=Counter(lst)
print(Counter(lst))
print(Counter(lst[1])) #frequency of each character in word
print(freq.most_common(3)) # Top 3 frequent characters
```

```
Counter({'Hello': 1, 'Randhir': 1, 'How': 1, 'Are': 1, 'You': 1})
Counter({'R': 1, 'a': 1, 'n': 1, 'd': 1, 'h': 1, 'i': 1, 'r': 1})
[('Hello', 1), ('Randhir', 1), ('How', 1)]
```

#Find common elements between two lists?

```
lst1=[2,3,4,1,2,3,7,8]
lst2=[5,6,2,9,8,6,5]
common=list(set(lst1)&set(lst2))
common
```

[8, 2]

#Simple Class & Object

```
class Dog:
    def sound(self):
        return "Woof"
class Cat:
    def sound(self):
        return "Meow"
def animal_sound(animal):
    print(animal.sound())
animal=[Dog(),Cat()]
for i in animal:
    animal_sound(i)
```

Woof
Meow

```
#Using super() in Inheritance
class student():
    def __init__(self,name):
        self.name=name
    def display(self):
        print(self.name)
class registration(student):
    def __init__(self,name,status):
        super().__init__(name)
        self.status=status
    def display(self):
        print(self.status)
        super().display()
r=registration("Randhir","Active")
r.display()
```

Active
Randhir

```
#Simulate Method Overloading using args
def mul(*args):
    result=1
    for i in args:
        result=result*i
    return result
print(mul(2,3,4))
print(mul(7,8))
```

24
56

```
# generator in python
def display(n):

    for i in range(1,n+1):
        yield i
for i in display(5):
    print(i)
```

1
2
3
4
5

```
def decorator(func):
    def wrapper():
```

```

        print("Before function call")
        func()
        print("After function call")
    return wrapper

```

```

@decorator
def say_hello():
    print("Hello World!")

```

```
say_hello()
```

```

Before function call
Hello World!
After function call

```

#Create a DataFrame and show basic operations

```

import pandas as pd
student=pd.DataFrame({"Name":['Randhir','Raj','Rajiv',None],
                      "Age":[18,None,32,50]})

print(student)
print(student.describe())
print(student.info())

```

```

      Name  Age
0  Randhir  18.0
1      Raj   NaN
2   Rajiv  32.0
3     None  50.0
      Age
count    3.000000
mean    33.333333
std     16.041613
min     18.000000
25%     25.000000
50%     32.000000
75%     41.000000
max     50.000000
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0   Name    3 non-null      object
 1   Age     3 non-null      float64
dtypes: float64(1), object(1)
memory usage: 196.0+ bytes
None

```

```
print(student.isnull().sum())
```

```
Name      1
Age        1
dtype: int64
```

#Add a new column based on existing columns

```
def age_gen(n):
    if n <=18:
        return "Young"
    elif n<=38:
        return "Middle"
    elif n <=90:
        return "Old"
    else:
        return "Invalid"
student['Gen']=student['Age'].apply(age_gen)
student
```

```
{"summary":{"\n  \"name\": \"student\",\n  \"rows\": 4,\n  \"fields\": [\n    {\n      \"column\": \"Name\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 3,\n        \"samples\": [\n          \"Randhir\",\n          \"Raj\",\n          \"Rajiv\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Age\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 16.041612554021285,\n        \"min\": 18.0,\n        \"max\": 50.0,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          18.0,\n          32.0,\n          50.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Gen\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          \"Invalid\",\n          \"Old\",\n          \"Young\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}, \"type\": \"dataframe\", \"variable_name\": \"student\"}
```

```
student['Age']=student['Age'].fillna(student['Age'].mean())
student
```

```
{"summary":{"\n  \"name\": \"student\",\n  \"rows\": 4,\n  \"fields\": [\n    {\n      \"column\": \"Name\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 3,\n        \"samples\": [\n          \"Randhir\",\n          \"Raj\",\n          \"Rajiv\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Age\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 13.097921802925667,\n        \"min\": 18.0,\n        \"max\": 50.0,\n        \"num_unique_values\": 4,\n        \"samples\": [\n          33.33333333333336,\n          50.0,\n          18.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Gen\",\n      \"properties\": {\n
```

```

\ "dtype\ ": \ "string\ ",\n          \ "num_unique_values\ ": 4,\n
\ "samples\ ": [\n          \ "Invalid\ ",\n          \ "Old\ ",\n
\ "Young\ "\n          ],\n          \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\ "\n          }\n          }\n          ]\n
n\ },\n          \ "type\ ": "dataframe",\n          \ "variable_name\ ": "student"}

#GroupBy and Aggregation (very common)
result=student.groupby('Gen')['Age'].agg(['mean','max','min','sum'])
result

{"summary":{"\n  \ "name\ ": \ "result\ ",\n  \ "rows\ ": 4,\n  \ "fields\ ":
[\n    {\n      \ "column\ ": \ "Gen\ ",\n      \ "properties\ ": {\n
\ "dtype\ ": \ "string\ ",\n      \ "num_unique_values\ ": 4,\n
\ "samples\ ": [\n      \ "Middle\ ",\n      \ "Young\ ",\n
\ "Invalid\ "\n      ],\n      \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\ "\n      }\n      },\n      {\n      \ "column\ ":
\ "mean\ ",\n      \ "properties\ ": {\n      \ "dtype\ ": \ "number\ ",\n
\ "std\ ": 13.097921802925667,\n      \ "min\ ": 18.0,\n      \ "max\ ":
50.0,\n      \ "num_unique_values\ ": 4,\n      \ "samples\ ": [\n
32.0,\n      18.0,\n      33.333333333333336\n      ],\n
\ "semantic_type\ ": \ "\",\n      \ "description\ ": \ "\ "\n      }\n
      },\n      {\n      \ "column\ ": \ "max\ ",\n      \ "properties\ ": {\n
\ "dtype\ ": \ "number\ ",\n      \ "std\ ": 13.097921802925667,\n
\ "min\ ": 18.0,\n      \ "max\ ": 50.0,\n      \ "num_unique_values\ ":
4,\n      \ "samples\ ": [\n      32.0,\n      18.0,\n
33.333333333333336\n      ],\n      \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\ "\n      }\n      },\n      {\n      \ "column\ ":
\ "min\ ",\n      \ "properties\ ": {\n      \ "dtype\ ": \ "number\ ",\n
\ "std\ ": 13.097921802925667,\n      \ "min\ ": 18.0,\n      \ "max\ ":
50.0,\n      \ "num_unique_values\ ": 4,\n      \ "samples\ ": [\n
32.0,\n      18.0,\n      33.333333333333336\n      ],\n
\ "semantic_type\ ": \ "\",\n      \ "description\ ": \ "\ "\n      }\n
      },\n      {\n      \ "column\ ": \ "sum\ ",\n      \ "properties\ ": {\n
\ "dtype\ ": \ "number\ ",\n      \ "std\ ": 13.097921802925667,\n
\ "min\ ": 18.0,\n      \ "max\ ": 50.0,\n      \ "num_unique_values\ ":
4,\n      \ "samples\ ": [\n      32.0,\n      18.0,\n
33.333333333333336\n      ],\n      \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\ "\n      }\n      }\n      }\n      ]\n
n\ },\n          \ "type\ ": "dataframe",\n          \ "variable_name\ ": "result"}

#Merge / Join two DataFrames
df2 = pd.DataFrame({'Name': ['Raj','Rajiv','Randhir'], 'Salary':
[70000,80000,90000]})
merge=pd.merge(student,df2,on='Name')
merge

{"summary":{"\n  \ "name\ ": \ "merge\ ",\n  \ "rows\ ": 3,\n  \ "fields\ ":
[\n    {\n      \ "column\ ": \ "Name\ ",\n      \ "properties\ ": {\n
\ "dtype\ ": \ "string\ ",\n      \ "num_unique_values\ ": 3,\n
\ "samples\ ": [\n      \ "Randhir\ ",\n      \ "Raj\ ",\n

```

```

{"Rajiv": 18.0, "Age": 33.33333333333336, "Gen": "Young", "Invalid": "Middle", "Salary": 70000.0, "Tax": 18000.0}, {"Rajiv": 32.0, "Age": 33.33333333333336, "Gen": "Invalid", "Invalid": "Middle", "Salary": 80000.0, "Tax": 14000.0}, {"Rajiv": 33.33333333333336, "Age": 33.33333333333336, "Gen": "Middle", "Invalid": "Middle", "Salary": 90000.0, "Tax": 14000.0}]
{"type": "dataframe", "variable_name": "merge"}

```

#Apply function row-wise or column-wise

```
merge['Tax'] = merge['Salary'].apply(lambda x: x * 0.20)
```

```
merge
```

```

{"summary": {"name": "merge", "rows": 3, "fields": [{"column": "Name", "properties": {"dtype": "string", "num_unique_values": 3, "samples": ["Randhir", "Raj", "Rajiv"]}, "semantic_type": "string", "description": "Name"}, {"column": "Age", "properties": {"dtype": "number", "std": 8.494006601607285, "min": 18.0, "max": 33.33333333333336, "num_unique_values": 3, "samples": [18.0, 33.33333333333336, 32.0]}, "semantic_type": "number", "description": "Age"}, {"column": "Gen", "properties": {"dtype": "string", "num_unique_values": 3, "samples": ["Young", "Invalid", "Middle"]}, "semantic_type": "string", "description": "Gen"}, {"column": "Salary", "properties": {"dtype": "number", "std": 10000.0, "min": 70000.0, "max": 90000.0, "num_unique_values": 3, "samples": [70000.0, 80000.0, 90000.0]}, "semantic_type": "number", "description": "Salary"}, {"column": "Tax", "properties": {"dtype": "number", "std": 2000.0, "min": 14000.0, "max": 18000.0, "num_unique_values": 3, "samples": [18000.0, 14000.0, 14000.0]}, "semantic_type": "number", "description": "Tax"}]}

```



```

16000.0\n          ],\n          \"semantic_type\": \"\", \n          \"description\": \"\"\n        }\n    }\n}\", \"type\": \"dataframe\", \"variable_name\": \"merge\"}

#Create different types of NumPy arrays
import numpy as np

arr1 = np.array([1, 2, 3, 4])           # 1D
arr2 = np.zeros((3, 3))                 # 3x3 zeros
arr3 = np.ones((2, 4))                  # 2x4 ones
arr4 = np.arange(0, 10, 2)              # [0 2 4 6 8]
arr5 = np.linspace(0, 1, 5)            # 5 evenly spaced
values
arr6 = np.random.rand(3, 2)            # random [0,1)
print(arr1.shape, arr2.ndim)           # shape and
dimensions

(4,) 2

a = np.arange(12).reshape(3, 4)        # reshape
print(a)

flat = a.flatten()                     # always copy
ravel = a.ravel()                      # may return view (faster)

trans = a.T                            # transpose

[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]

#Element-wise Operations & Broadcasting
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
print(a + b)                           # [11 22 33]
print(a * 5)                           # broadcasting

# 2D broadcasting example
mat = np.array([[1,2],[3,4]])
print(mat + 10)

[[11 22 33]
 [ 5 10 15]
 [[11 12]
 [13 14]]

arr = np.array([1, 2, 3, 4, 5, 100])

print(np.mean(arr))
print(np.median(arr))
print(np.std(arr))

```

```
print(np.sum(arr))
print(np.min(arr), np.max(arr))

19.166666666666668
3.5
36.17281053805775
115
1 100

a = np.array([1, 2])
b = np.array([3, 4])

print(np.vstack((a, b))) # vertical stack
print(np.hstack((a, b))) # horizontal stack

[[1 2]
 [3 4]]
[1 2 3 4]
```